Hospital Medicine-Delivery Robot

Assignment 1 (CO1) - Python source code

1. hospital_search.py - Graph generator + BFS / UCS / IDDFS (Tasks 1, 8)
2. run_experiments.py - Tests, performance data, best/avg/worst cases (Tasks 4, 8, 9)
3. belief_search.py - Day-4 challenge: belief-state search (Task 12)
4. make_plots.py - Graphs and growth exponents (Tasks 9, 10)

Run order: python run_experiments.py ; python belief_search.py ; python make_plots.py (Python 3 + NumPy + Matplotlib)

hospital_search.py - Graph generator + BFS / UCS / IDDFS (Tasks 1, 8)

```
"""
hospital_search.py  --  Hospital medicine-delivery robot: graph generator + 4 uninformed searches
Algorithms:
  1. bfs            : iterative, FIFO queue               (fewest moves)
  2. ucs            : iterative, priority queue           (least travel-time)
  3. iddfs          : RECURSIVE depth-limited DFS repeated for limits 0,1,2,...
                      (graph-search version: depth-labelled duplicate detection)
  3b. iddfs_path    : same recursion but only cycle-checks the current path (O(b*m)-memory tree-search style)
Every algorithm returns the same dictionary so that results can be compared fairly.
"""
import sys, heapq
from collections import deque
import numpy as np

sys.setrecursionlimit(1_000_000)
INF = float("inf")


# ------------------------------------------------------------------------------------
# Graph generation + preprocessing (CSR adjacency list).  Cost is Theta(V + E).
# ------------------------------------------------------------------------------------
class Graph:
    def __init__(self, n, start, nbr, wt, n_edges):
        self.n, self.start, self.nbr, self.wt, self.n_edges = n, start, nbr, wt, n_edges

    def edges_of(self, u):
        s, e = self.start[u], self.start[u + 1]
        return zip(self.nbr[s:e], self.wt[s:e])


def build_hospital_graph(n, seed, extra_ratio=0.5):
    """n connected 'locations' + 2 extra ISOLATED locations (ids n, n+1) = an unreachable closed wing.
    Spanning tree (random recursive tree) + extra_ratio*n random shortcut corridors; weights 1..10 minutes."""
    rng = np.random.default_rng(seed)
    child = np.arange(1, n)
    parent = (rng.random(n - 1) * child).astype(np.int64)          # parent of node i is uniform in [0,i)
    m_extra = int(extra_ratio * n)
    a = rng.integers(0, n, m_extra)
    b = rng.integers(0, n, m_extra)
    keep = a != b
    u = np.concatenate([parent, a[keep]])
    v = np.concatenate([child, b[keep]])
    lo, hi = np.minimum(u, v), np.maximum(u, v)
    key = lo * (n + 2) + hi
    key, first = np.unique(key, return_index=True)                  # remove duplicate corridors
    lo, hi = lo[first], hi[first]
    w = rng.integers(1, 11, len(lo))
    # closed wing: edge between n and n+1
    lo = np.append(lo, n); hi = np.append(hi, n + 1); w = np.append(w, 3)
    N = n + 2
    src = np.concatenate([lo, hi]); dst = np.concatenate([hi, lo]); ww = np.concatenate([w, w])
    order = np.argsort(src, kind="stable")
    src, dst, ww = src[order], dst[order], ww[order]
    start = np.zeros(N + 1, dtype=np.int64)
    np.add.at(start, src + 1, 1)
    start = np.cumsum(start)
    return Graph(N, start.tolist(), dst.tolist(), ww.tolist(), len(lo))


# ------------------------------------------------------------------------------------
# Helpers
# ------------------------------------------------------------------------------------
def _result(found, path, cost, gen, exp, mf, aux=0, extra=None):
    r = dict(found=found, path=path, cost=cost, generated=gen, expanded=exp, max_frontier=mf, aux_memory=aux,
             depth=(len(path) - 1) if path else None)
    if extra:
        r.update(extra)
    return r


def _rebuild(parent, t):
    path = []
    c = t
    while c is not None:
        path.append(c)
        c = parent[c][0]
    path.reverse()
    cost = sum(parent[x][1] for x in path[1:])
    return path, cost
```

hospital_search.py - Graph generator + BFS / UCS / IDDFS (Tasks 1, 8) (cont.)

```
# ------------------------------------------------------------------------------------
# 1. Breadth-first search (iterative, FIFO queue, early goal test, explored set = dict)
# ------------------------------------------------------------------------------------
def bfs(g, s, t):
    if s == t:
        return _result(True, [s], 0, 1, 0, 1)
    parent = {s: (None, 0)}
    q = deque([s]); gen, exp, mf = 1, 0, 1
    nbr, wt, st = g.nbr, g.wt, g.start
    while q:
        u = q.popleft(); exp += 1
        for i in range(st[u], st[u + 1]):
            v = nbr[i]; gen += 1
            if v not in parent:
                parent[v] = (u, wt[i])
                if v == t:
                    p, c = _rebuild(parent, t)
                    return _result(True, p, c, gen, exp, mf, len(parent))
                q.append(v)
        if len(q) > mf: mf = len(q)
    return _result(False, None, None, gen, exp, mf, len(parent))


# ------------------------------------------------------------------------------------
# 2. Uniform-cost search (iterative, binary heap, goal test on expansion, lazy deletion)
# ------------------------------------------------------------------------------------
def ucs(g, s, t):
    best = {s: 0}; parent = {s: (None, 0)}
    heap = [(0, 0, s)]; tick = 1
    gen, exp, mf = 1, 0, 1
    nbr, wt, st = g.nbr, g.wt, g.start
    done = set()
    while heap:
        c, _, u = heapq.heappop(heap)
        if u in done:
            continue                                  # stale entry
        if u == t:
            p, cost = _rebuild(parent, t)
            return _result(True, p, cost, gen, exp, mf, len(best))
        done.add(u); exp += 1
        for i in range(st[u], st[u + 1]):
            v = nbr[i]; gen += 1
            nc = c + wt[i]
            if nc < best.get(v, INF):
                best[v] = nc; parent[v] = (u, wt[i])
                heapq.heappush(heap, (nc, tick, v)); tick += 1
        if len(heap) > mf: mf = len(heap)
    return _result(False, None, None, gen, exp, mf, len(best))


# ------------------------------------------------------------------------------------
# 3. Iterative-deepening DFS : RECURSIVE depth-limited search (DLS), limits 0,1,2,...
#    dup='depth' -> remember the shallowest depth at which each node was reached in this
#                   iteration, re-enter only if reached shallower (correct + bounded by O(V+E) per limit)
#    dup='path'  -> only forbid revisiting nodes on the CURRENT path (memory O(m), may revisit nodes
#                   through different routes => tree-like blow-up).  `cap` stops runaway runs.
# ------------------------------------------------------------------------------------
def iddfs(g, s, t, dup="depth", cap=None):
    nbr, wt, st = g.nbr, g.wt, g.start
    gen = exp = 0
    max_rec = 0
    aux_peak = 0
    class Capped(Exception):
        pass

    limit = 0
    while True:
        cutoff = False
        best = {s: 0} if dup == "depth" else None
        onpath = {s} if dup == "path" else None
        path = [s]; costs = [0]

        def dls(u, d):                        # <- recursive subproblem: remaining depth = limit - d
            nonlocal gen, exp, cutoff, max_rec
            if u == t:
                return True
            if d == limit:
```

hospital_search.py - Graph generator + BFS / UCS / IDDFS (Tasks 1, 8) (cont.)

```
                cutoff = True
                return False
            exp += 1
            if len(path) > max_rec: max_rec = len(path)
            for i in range(st[u], st[u + 1]):
                v = nbr[i]; gen += 1
                if cap and gen > cap:
                    raise Capped
                if dup == "depth":
                    if best.get(v, INF) <= d + 1:
                        continue
                    best[v] = d + 1
                else:
                    if v in onpath:
                        continue
                    onpath.add(v)
                path.append(v); costs.append(costs[-1] + wt[i])
                if dls(v, d + 1):
                    return True
                path.pop(); costs.pop()
                if dup == "path":
                    onpath.discard(v)
            return False

        try:
            ok = dls(s, 0)
        except Capped:
            return _result(False, None, None, gen, exp, max_rec, aux_peak, dict(capped=True, limit_reached=limit))
        if best is not None and len(best) > aux_peak: aux_peak = len(best)
        if ok:
            return _result(True, list(path), costs[-1], gen, exp, max_rec, aux_peak, dict(final_limit=limit))
        if best is not None:
            # depth-labelled mode: after a full iteration best[v] = true shortest hop-distance of every node
            # within `limit`; if no node sits exactly at distance `limit`, nothing deeper exists -> exhausted.
            cutoff = max(best.values()) == limit
        if not cutoff:                      # whole reachable graph exhausted without hitting the limit
            return _result(False, None, None, gen, exp, max_rec, aux_peak, dict(final_limit=limit))
        limit += 1


def iddfs_path(g, s, t, cap=3_000_000):
    return iddfs(g, s, t, dup="path", cap=cap)


ALGOS = {"BFS": bfs, "UCS": ucs, "IDDFS": iddfs, "IDDFS-path": iddfs_path}


def validate(g, path, s, t):
    """Independent route check: starts at s, ends at t, each hop is a real corridor."""
    if not path or path[0] != s or path[-1] != t:
        return False
    for a, b in zip(path, path[1:]):
        lo, hi = g.start[a], g.start[a + 1]
        if b not in g.nbr[lo:hi]:
            return False
    return True
```

run_experiments.py - Tests, performance data, best/avg/worst cases (Tasks 4, 8, 9)

```
"""run_experiments.py -- Tasks 8, 9, 4 (tests, performance data, best/avg/worst).  Reproducible: all seeds fixed."
""
import time, json, random, statistics as stt, sys
from hospital_search import *

GRAPH_SEED = 2026            # seed for graph generation (graph for size n uses GRAPH_SEED + n)
GOAL_SEED = 7                # seed for choosing the 5 requested wards
SIZES = [1_000, 10_000, 100_000, 1_000_000]
START = 0                    # central pharmacy
NGOALS = 5
out = {}


def log(*a):
    print(*a, flush=True)


# ---------------------------------------------------------------------- TASK 8 : correctness tests
log("=" * 78); log("TASK 8 - CORRECTNESS TESTS"); log("=" * 78)
# (a) hand-made graph where fewest-moves path != cheapest path  (cheap-edge detour)
#     0 --10--> 4 (direct), 0-1-2-3-4 each cost 1  -> cheapest = 4, fewest moves = 1 hop
edges = [(0, 4, 10), (0, 1, 1), (1, 2, 1), (2, 3, 1), (3, 4, 1)]
N = 5
adj = [[] for _ in range(N)]
for a, b, w in edges:
    adj[a].append((b, w)); adj[b].append((a, w))
start = [0]; nbr = []; wt = []
for u in range(N):
    for v, w in adj[u]:
        nbr.append(v); wt.append(w)
    start.append(len(nbr))
tiny = Graph(N, start, nbr, wt, len(edges))
log("Tiny graph (direct corridor 0-4 costs 10; detour 0-1-2-3-4 costs 4):")
for name, f in ALGOS.items():
    r = f(tiny, 0, 4)
    log(f"  {name:11s} path={r['path']} hops={r['depth']} cost={r['cost']}  valid={validate(tiny, r['path'], 0, 4)
}")

# (b) generated 1,000-state hospital: reachable / start==goal / unreachable
g1k = build_hospital_graph(1000, GRAPH_SEED + 1000)
rng = random.Random(GOAL_SEED)
far_goal = rng.randrange(1, 1000)
log(f"\n1,000-state hospital (V={g1k.n}, undirected corridors={g1k.n_edges}), graph seed={GRAPH_SEED+1000}")
tests = [("reachable", START, far_goal), ("start==goal", 17, 17), ("unreachable (closed wing)", START, 1000)]
for label, s, t in tests:
    log(f"\n  Test: {label}   start={s} goal={t}")
    ref = ucs(g1k, s, t)
    for name, f in ALGOS.items():
        r = f(g1k, s, t)
        if r["found"]:
            ok = validate(g1k, r["path"], s, t)
            opt = (r["cost"] == ref["cost"])
            log(f"    {name:11s} found hops={r['depth']:2d} cost={r['cost']:3d} valid={ok} cost-optimal={opt} gen=
{r['generated']} exp={r['expanded']}")
        else:
            log(f"    {name:11s} NOT FOUND (correct={not ref['found']})  gen={r['generated']} exp={r['expanded']}"
)
    # cross-checks
    b_, u_, i_ = bfs(g1k, s, t), ucs(g1k, s, t), iddfs(g1k, s, t)
    assert b_["found"] == u_["found"] == i_["found"] == (label != "unreachable (closed wing)")
    if b_["found"]:
        assert b_["depth"] == i_["depth"] <= u_["depth"]
        assert u_["cost"] <= b_["cost"] and u_["cost"] <= i_["cost"]
log("\nAll correctness assertions passed (found/not-found agree; BFS hops == IDDFS hops; UCS cost <= others).")

# ---------------------------------------------------------------------- TASK 9 : performance on 4 sizes
log("\n" + "=" * 78); log("TASK 9 - PERFORMANCE (start=0 pharmacy; 5 reachable wards per size; mean of 5 runs)");
log("=" * 78)
rows = []
for n in SIZES:
    t0 = time.perf_counter()
    g = build_hospital_graph(n, GRAPH_SEED + n)
    prep = time.perf_counter() - t0
    rng = random.Random(GOAL_SEED + n)
    goals = [rng.randrange(1, n) for _ in range(NGOALS)]
    # skip goals identical to start
    log(f"\nn={n:>9,}  V={g.n:,}  E(undirected)={g.n_edges:,}  preprocessing(build CSR adjacency)={prep:.2f}s  goa
ls={goals}")
    ref_costs = {}
```

run_experiments.py - Tests, performance data, best/avg/worst cases (Tasks 4, 8, 9) (cont.)

```
    for name, f in ALGOS.items():
        recs = []
        for gl in goals:
            t1 = time.perf_counter()
            r = f(g, START, gl)
            dt = time.perf_counter() - t1
            r["time"] = dt
            if name == "UCS":
                ref_costs[gl] = r["cost"]
            recs.append((gl, r))
        # optimality vs UCS
        row = dict(n=n, algo=name, prep=prep)
        row["found"] = sum(1 for _, r in recs if r["found"])
        done = [(gl, r) for gl, r in recs if r["found"]]
        for key in ["generated", "expanded", "max_frontier", "aux_memory", "time"]:
            row[key] = stt.mean(r[key] for _, r in recs)
        if done:
            row["depth"] = stt.mean(r["depth"] for _, r in done)
            row["cost"] = stt.mean(r["cost"] for _, r in done)
            row["optimal"] = sum(1 for gl, r in done if name != "UCS" and ref_costs.get(gl) is not None and r["cos
t"] == ref_costs[gl]) \
                if name != "UCS" else len(done)
            row["valid"] = all(validate(g, r["path"], START, gl) for gl, r in done)
        row["capped"] = sum(1 for _, r in recs if r.get("capped"))
        rows.append(row)
        log(f"  {name:11s} found={row['found']}/{NGOALS} capped={row['capped']} gen={row['generated']:>12,.0f} exp
={row['expanded']:>10,.0f} "
            f"maxFrontier={row['max_frontier']:>9,.0f} aux={row['aux_memory']:>9,.0f} "
            + (f"depth={row['depth']:.1f} cost={row['cost']:.1f} optimal={row['optimal']}/{row['found']} valid={ro
w['valid']} " if done else "")
            + f"time={row['time']:.3f}s")
    del g
out["performance"] = rows

# ------------------------------------------------------------------ TASK 4 : best / average / worst
log("\n" + "=" * 78); log("TASK 4 - BEST / AVERAGE / WORST CASE (nodes generated)"); log("=" * 78)
cases = []
for n in [10_000, 100_000]:
    g = build_hospital_graph(n, GRAPH_SEED + n)
    nb = g.nbr[g.start[START]:g.start[START + 1]]
    best_goal = nb[0]                                   # goal adjacent to pharmacy (depth 1)
    # deepest node by BFS hops = "worst reachable"
    dist = {START: 0}; q = deque([START]); last = START
    while q:
        u = q.popleft(); last = u
        for v, _ in g.edges_of(u):
            if v not in dist: dist[v] = dist[u] + 1; q.append(v)
    worst_goal = last
    unreach = n
    rng = random.Random(99)
    avg_goals = [rng.randrange(1, n) for _ in range(5)]
    log(f"\nn={n:,}: best goal={best_goal} (depth {dist[best_goal]}), deepest reachable goal={worst_goal} (depth {
dist[worst_goal]}), unreachable={unreach}")
    for name, f in ALGOS.items():
        b = f(g, START, best_goal)["generated"]
        a = stt.mean(f(g, START, x)["generated"] for x in avg_goals)
        w = f(g, START, worst_goal)["generated"]
        u = f(g, START, unreach)
        log(f"  {name:11s} best={b:>10,}  average={a:>12,.0f}  deepest-reachable={w:>12,}  unreachable={u['generat
ed']:>12,}" + (" (CAPPED)" if u.get("capped") else ""))
        cases.append(dict(n=n, algo=name, best=b, avg=a, deepest=w, unreachable=u["generated"], capped=bool(u.get(
"capped"))))
    del g
out["cases"] = cases
json.dump(out, open("results.json", "w"), indent=1)
log("\nSaved results.json")
```

belief_search.py - Day-4 challenge: belief-state search (Task 12)

```
"""belief_search.py -- Task 12 (Day-4 challenge): robot only knows it is in one of k junctions.
Hospital floor = W x H grid of junctions with some blocked cells; actions U,D,L,R (a move into a wall = stay).
Dock (pharmacy docking station) at (0,0).  Goal: be SURE the robot is at the dock  (belief == {dock}).

 A) retain original design : plan from one guessed start, execute it blindly  -> success rate
 B) sensorless belief-state BFS : state = frozenset of possible junctions, action maps set -> image set
 C) belief-state AND-OR search with a LOCAL wall sensor (percept = which of U,D,L,R are blocked):
    after every move the belief splits by percept; plan is a conditional plan (tree); we minimise its
    worst-case length by iterative deepening on the plan depth with memoisation.
"""
import random, itertools, statistics as stt, math
from collections import deque

W = H = 8
DIRS = {"U": (-1, 0), "D": (1, 0), "L": (0, -1), "R": (0, 1)}
ACTS = list(DIRS)


def make_grid(seed, density=0.15):
    rng = random.Random(seed)
    free = {(r, c) for r in range(H) for c in range(W) if rng.random() > density}
    free.add((0, 0))
    # keep only the component containing the dock
    comp, q = {(0, 0)}, deque([(0, 0)])
    while q:
        r, c = q.popleft()
        for dr, dc in DIRS.values():
            nb = (r + dr, c + dc)
            if nb in free and nb not in comp:
                comp.add(nb); q.append(nb)
    return frozenset(comp)


class Floor:
    def __init__(self, cells):
        self.cells = cells
        self.dock = (0, 0)
        self.move = {(x, a): self._m(x, a) for x in cells for a in ACTS}
        self.percept = {x: tuple(self.move[(x, a)] == x for a in ACTS) for x in cells}

    def _m(self, x, a):
        dr, dc = DIRS[a]
        nb = (x[0] + dr, x[1] + dc)
        return nb if nb in self.cells else x

    def image(self, belief, a):
        return frozenset(self.move[(x, a)] for x in belief)

    def split(self, belief):
        parts = {}
        for x in belief:
            parts.setdefault(self.percept[x], set()).add(x)
        return [frozenset(p) for p in parts.values()]


# ---- A) original design: BFS plan for ONE guessed start, then executed blindly --------------------
def plan_single(fl, s):
    prev = {s: None}; q = deque([s])
    while q:
        u = q.popleft()
        if u == fl.dock:
            break
        for a in ACTS:
            v = fl.move[(u, a)]
            if v not in prev:
                prev[v] = (u, a); q.append(v)
    acts = []; c = fl.dock
    while prev[c] is not None:
        c, a = prev[c][0], prev[c][1]
        acts.append(a); c = c
    # (reconstruct properly)
    acts = []; c = fl.dock
    while prev[c] is not None:
        u, a = prev[c]; acts.append(a); c = u
    return acts[::-1]


def blind_success(fl, belief, guess):
    plan = plan_single(fl, guess)
```

belief_search.py - Day-4 challenge: belief-state search (Task 12) (cont.)

```
    ok = 0
    for x in belief:
        for a in plan:
            x = fl.move[(x, a)]
        ok += (x == fl.dock)
    return ok / len(belief), len(plan)


# ---- B) sensorless belief-state BFS ------------------------------------------------------------
def sensorless(fl, b0, cap=2_000_000):
    goal = frozenset({fl.dock})
    if b0 == goal:
        return dict(solved=True, length=0, generated=1, expanded=0)
    prev = {b0: None}; q = deque([b0]); gen = 1; exp = 0
    while q:
        b = q.popleft(); exp += 1
        for a in ACTS:
            nb = fl.image(b, a); gen += 1
            if nb not in prev:
                prev[nb] = (b, a)
                if nb == goal:
                    L = 0; c = nb
                    while prev[c] is not None: c = prev[c][0]; L += 1
                    return dict(solved=True, length=L, generated=gen, expanded=exp, distinct=len(prev))
                if len(prev) > cap:
                    return dict(solved=False, length=None, generated=gen, expanded=exp, capped=True)
                q.append(nb)
    return dict(solved=False, length=None, generated=gen, expanded=exp, distinct=len(prev))


# ---- C) belief-state AND-OR search with local sensing ------------------------------------------
def sensed(fl, b0, max_depth=60):
    goal = frozenset({fl.dock})
    memo = {}
    seen = set()
    calls = [0]

    def can(b, D):
        # True iff a conditional plan of worst-case length <= D exists for belief b (b already refined by percept)
        if b == goal: return True
        if D == 0: return False
        key = (b, D)
        if key in memo: return memo[key]
        calls[0] += 1; seen.add(b)
        res = False
        for a in ACTS:
            nb = fl.image(b, a)
            if nb == b: continue
            if all(can(p, D - 1) for p in fl.split(nb)):
                res = True; break
        memo[key] = res
        return res

    parts = fl.split(b0)
    for D in range(0, max_depth + 1):
        if all(can(p, D) for p in parts):
            return dict(solved=True, length=D, generated=len(seen), calls=calls[0])
    return dict(solved=False, length=None, generated=len(seen), calls=calls[0])


def run(ks=(2, 4, 8), trials=8, base_seed=100):
    rows = []
    for k in ks:
        A, B, C = [], [], []
        for t in range(trials):
            cells = make_grid(base_seed + t)
            fl = Floor(cells)
            rng = random.Random(1000 * k + t)
            cand = sorted(c for c in cells if c != fl.dock)
            b0 = frozenset(rng.sample(cand, k))
            A.append(blind_success(fl, b0, min(b0))[0])
            B.append(sensorless(fl, b0))
            C.append(sensed(fl, b0))
        sol = [r for r in B if r["solved"]]; solc = [r for r in C if r["solved"]]
        rows.append(dict(k=k,
                         blind=stt.mean(A),
                         sl_solved=len(sol), sl_gen=stt.mean(r["generated"] for r in sol) if sol else None,
                         sl_len=stt.mean(r["length"] for r in sol) if sol else None,
                         se_solved=len(solc), se_gen=stt.mean(r["generated"] for r in solc) if solc else None,
```

belief_search.py - Day-4 challenge: belief-state search (Task 12) (cont.)

```
                        se_len=stt.mean(r["length"] for r in solc) if solc else None,
                        trials=trials))
    return rows


if __name__ == "__main__":
    print("TASK 12 D - Evidence (8x8 floor, 15% blocked cells, dock at (0,0), 8 random floors/belief sets per k)")
    print("Floor seeds 100..107; belief sets sampled with random.Random(1000*k+trial)\n")
    rows = run()
    print("Columns: blind = success rate of the ORIGINAL plan from one guessed start;")
    print("         SL = sensorless belief-state BFS; SE = belief-state AND-OR search with local sensor")
    print("         solved = floors solved out of 8; gen = belief states generated; len = plan length (SE: worst c
ase)\n")
    hdr = f"{'k':>2} {'blind':>7} | {'SL solved':>9} {'SL gen':>11} {'SL len':>7} | {'SE solved':>9} {'SE gen':>7}
 {'SE len':>7}"
    print(hdr); print("-" * len(hdr))
    f = lambda x, p=1: "n/a" if x is None else f"{x:.{p}f}"
    for r in rows:
        print(f"{r['k']:>2} {r['blind']*100:>6.1f}% | {r['sl_solved']:>7}/{r['trials']} {f(r['sl_gen'],0):>11} {f(
r['sl_len']):>7} | "
              f"{r['se_solved']:>7}/{r['trials']} {f(r['se_gen']):>7} {f(r['se_len']):>7}")
    print("\nSize of the belief space (number of possible 'knowledge states'):")
    for n in (64, 1000, 10**6):
        print(f"  n={n:>9,}: " + "  ".join(f"C(n,{k})={math.comb(n,k):.3e}" for k in (2, 4, 8)) + f"   2^n-1 = {'a
stronomical' if n>64 else f'{2**n-1:.3e}'}")
```

make_plots.py - Graphs and growth exponents (Tasks 9, 10)

```
import json, numpy as np, matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
R = json.load(open("results.json"))["performance"]
algos = ["BFS","UCS","IDDFS","IDDFS-path"]
col = {"BFS":"#1f77b4","UCS":"#d62728","IDDFS":"#2ca02c","IDDFS-path":"#9467bd"}
def series(a,k): return [r["n"] for r in R if r["algo"]==a], [r[k] for r in R if r["algo"]==a]
def plot(key, title, ylabel, fn, ref=True):
    plt.figure(figsize=(6.2,4.2))
    for a in algos:
        x,y = series(a,key); y=[max(v,0.5) for v in y]
        plt.plot(x,y,"o-",label=a,color=col[a])
    if ref:
        x=np.array([1e3,1e6]); y0=series("BFS",key)[1][0]
        plt.plot(x,y0*x/1e3,"k:",label="linear in n (reference)")
    plt.xscale("log"); plt.yscale("log"); plt.xlabel("number of states n"); plt.ylabel(ylabel); plt.title(title)
    plt.grid(True,which="both",alpha=.3); plt.legend(fontsize=8); plt.tight_layout(); plt.savefig(fn,dpi=160); plt
.close()
plot("generated","Nodes generated vs state-space size","nodes generated (mean of 5 goals)","fig_generated.png")
plot("max_frontier","Max frontier / stack size (memory proxy)","max frontier size","fig_frontier.png")
plot("time","Execution time vs state-space size","seconds (mean of 5 runs)","fig_time.png")
plot("aux_memory","Auxiliary memory (explored / depth-label entries)","entries held","fig_aux.png")
print("log-log slopes n:1e3->1e6")
for k in ["generated","expanded","max_frontier","aux_memory","time"]:
    for a in algos:
        x,y=series(a,k); s=np.polyfit(np.log10(x),np.log10(np.maximum(y,1e-9)),1)[0]
        print(f"{k:13s} {a:11s} slope={s:.2f}  values={[round(v,3) if k=='time' else int(v) for v in y]}")
for r in R:
    if r["algo"]=="BFS": print(r["n"],"prep",round(r["prep"],3),"depth",r["depth"])
```